

Lab 10 · Cuando el otro organismo no contesta

Curso de Spring Boot · Servicio de Impuestos Internos · 2026

75 minutos · Spring Boot 4.1.0 · Java 25 (Temurin)

Resumen

Que depender de otro servicio es depender de que conteste. Medido de punta a punta:
30,15 s esperando · 2,21 s con timeout · 0,0018 s con el circuito abierto — y las llamadas a la red congeladas mientras el otro se recupera.

Índice

1. Antes de empezar	2
1.1. Qué vas a lograr	2
1.2. Qué necesitas tener listo	2
1.3. Cómo copiar el código de esta guía	3
1.4. La puesta a punto	3
2. El caso	3
2.1. La llamada telefónica, que es la metáfora de este laboratorio	3
3. Los pasos	3
3.1. Paso 1 · La espera infinita	3
3.2. Paso 2 · Colgar a los dos segundos	4
3.3. Paso 3 · Reintentar, y cuándo no	6
3.4. Paso 4 · Dejar de llamar — el circuito	7
3.5. Paso 5 · Contestar aunque no haya respuesta	9
4. Lo que aprendiste	11
5. Para profundizar	11
6. Antes de cerrar	11

1 Antes de empezar

1.1 Qué vas a lograr

Tu aplicación llama a Tesorería para consultar un pago. Hoy vas a ver qué pasa cuando **Tesorería no contesta**, y a descubrir que el problema no es de Tesorería: es tuyo, porque tus usuarios se quedan esperando con ella.

Vas a medir la espera, cortarla, reintentar cuando tenga sentido, **dejar de llamar** a quien está caído, y finalmente contestar algo útil aunque no haya respuesta.

1.2 Qué necesitas tener listo

Requisito	Cómo lo compruebas	Qué tiene que salir
Los labs 01 y 02 hechos	Sabes crear endpoints y pedir dependencias	—
Estar en la carpeta del lab	cd labs/lab-10-resiliencia/practica	El cd no da error
Ninguna base de datos	Este lab no la usa	—

Tesorería no existe: la simula la propia aplicación. Dentro corre un servidor de mentira que puedes poner sano, lento o caído con un curl. No hay que arrancar nada aparte, ni tener Docker.

1.3 Cómo copiar el código de esta guía

Al copiar de un PDF se pierden los espacios del principio de línea, y a veces una línea larga se parte en dos. Con Java no importa. El código completo está en `labs/lab-10-resiliencia/solucion/`.

1.4 La puesta a punto

```
cd labs/lab-10-resiliencia/practica
./mvnw spring-boot:run
```

Escucha en el **8097**. Vas a necesitar **dos terminales**: una con la aplicación y otra para los `curl`.

2 El caso

Para saber si un trámite está pagado, la DGT **llama a Tesorería**, que es otro organismo. Cuando Tesorería va bien, no se nota. Cuando va mal, se nota en la DGT.

2.1 La llamada telefónica, que es la metáfora de este laboratorio

Llamas a Tesorería por teléfono, y hay alguien esperando en tu ventanilla.

Mientras estás al teléfono, **el de tu ventanilla espera contigo**. Si Tesorería tarda medio minuto en coger, tu ciudadano espera medio minuto — y tú no puedes atender a nadie más. Cuatro decisiones, y son las cuatro del laboratorio:

1. **Colgar a los dos segundos**. Si no cogen, no te quedas escuchando el tono. Es el **timeout**.
2. **Volver a marcar una vez más**. A veces es la línea, no el organismo. Es el **reintento**.
3. **Dejar de llamar un rato**. Si llevas cinco llamadas seguidas sin respuesta, Tesorería está caída: seguir marcando no la arregla, **y encima la entorpece cuando intenta levantarse**. Es el **circuit breaker**.
4. **Atender igual, con lo que sabes**. «No he podido confirmar el pago; su trámite sigue su curso». Es la **degradación**, y es la única de las cuatro que es una decisión de negocio y no técnica.

3 Los pasos

3.1 Paso 1 · La espera infinita

Qué vamos a hacer

Poner Tesorería lenta y **cronometrar** lo que espera el usuario.

Para entenderlo mejor

Marcar, y quedarse escuchando el tono. Sin colgar.

El problema

Un cliente HTTP **sin timeout no tiene límite**. Si el otro lado no contesta, tu hilo se queda ahí — y los hilos son un recurso finito. Con suficientes peticiones colgadas, **tu aplicación deja de**

atender a todo el mundo, incluidos los que no tenían nada que ver con Tesorería.

Se corre

```
curl -X POST localhost:8097/simulador/lenta  
curl -s -o /dev/null -w '%{time_total}s\n' localhost:8097/pagos/77
```

Lo que vas a ver

```
30.157280s
```

Treinta segundos. Y no es que Tesorería tarde treinta: es que **tú estás dispuesto a esperar para siempre**, y ella tardó treinta.

Vas bien si...

El `curl` tarda unos 30 segundos. Si tardara dos, ya tienes el timeout del paso 2 puesto.

Si te atascas

1 · Port 8097 was already in use

Tienes la aplicación arrancada en otra terminal, o quedó viva. `Ctrl+C` en la otra, o:

```
lsof -ti:8097 | xargs kill -9
```

2 · El `curl` responde al instante con un pago normal.

No llegó el `/simulador/lenta`, o alguien lo dejó sano. Vuelve a mandarlo y mira que devuelva algo.

3.2 Paso 2 · Colgar a los dos segundos

Qué vamos a hacer

Ponerle timeout de conexión y de lectura al cliente HTTP.

Para entenderlo mejor

Colgar. Si en dos segundos no han cogido, no van a coger.

El problema

Sin límite, el que decide cuánto esperas es **el otro**. Y el otro puede estar caído, saturado, o detrás de una red que se tragó el paquete.

La alternativa, y por qué no

- **No poner timeout:** es el valor por defecto de casi todos los clientes, y es una bomba de relojería.
- **Un timeout muy largo (30 s):** parece prudente y no resuelve nada — sólo tarda más en fallar.

- **Dos segundos**, que es lo de aquí: el criterio no es «cuánto tarda normalmente», sino **cuánto está dispuesto a esperar tu usuario**. Si Tesorería normalmente responde en 200 ms, dos segundos ya es diez veces su normalidad.

Y una decisión más, que no se ve pero decide la clase: el transporte se fija a mano. WireMock —que simula Tesorería— arrastra Apache HttpClient al classpath, Spring lo prefiere si lo encuentra, y **Apache reintentará por su cuenta**. Este lab cuenta llamadas y mide tiempos: un reintento invisible falsearía las dos medidas.

Se pega

En `practica/src/main/java/cl/dgt/resiliencia/tesoreria/ClienteTesoreria.java`, **arriba con los import:**

```
import org.springframework.http.client.JdkClientHttpRequestFactory;

import java.net.http.HttpClient;
import java.time.Duration;
```

Las dos constantes, **arriba del todo de la clase**, y el constructor **entero**:

```
private static final Duration TIMEOUT_CONEXION = Duration.ofSeconds(2);
private static final Duration TIMEOUT_LECTURA = Duration.ofSeconds(2);

private final RestClient http;
private final AtomicInteger llamadas = new AtomicInteger();

public ClienteTesoreria(@Value("${lab09.tesoreria.puerto}") int puerto) {
    JdkClientHttpRequestFactory fabrica = new JdkClientHttpRequestFactory(
        HttpClient.newBuilder().connectTimeout(TIMEOUT_CONEXION).build());
    fabrica.setReadTimeout(TIMEOUT_LECTURA);

    this.http = RestClient.builder()
        .baseUrl("http://localhost:" + puerto)
        .requestFactory(fabrica)
        .build();
}
```

Lo que vas a ver

2.212154s

De 30 segundos a 2. El hilo se liberó y la aplicación sigue atendiendo a todos los demás.

Y ahora la mitad incómoda del paso, que es la que no hay que saltarse: falla **rápido**. Pero falla. El usuario ya no espera medio minuto — recibe un error en dos segundos. Un timeout es la primera medida, no la última.

Vas bien si...

El curl tarda unos 2 segundos en vez de 30, y devuelve un error.

Si te atascas

1 · Sigue tardando 30 segundos.

No reiniciaste, o el timeout está puesto en el sitio equivocado. Son **dos**: el de conexión va en el `HttpClient`, el de lectura en la fábrica.

2 · Tarda 30 s aunque el timeout está bien puesto.

Es el caso de Apache: si no fijaste el transporte a mano, Spring puede haber elegido otro cliente que ignora tu configuración. Comprueba que está el `JdkClientHttpRequestFactory`.

3.3 Paso 3 · Reintentar, y cuándo no

Qué vamos a hacer

Volver a intentarlo un par de veces antes de rendirse.

Para entenderlo mejor

Volver a marcar. Muchos fallos de red son de un instante: un paquete perdido, un reinicio de un segundo. Marcar otra vez cuesta poco y salva esos casos.

El problema

Un fallo transitorio no distingue: se ve igual que uno permanente. Rendirse al primero convierte un parpadeo en un error para el usuario.

La alternativa, y por qué no

- **No reintentar**: cualquier parpadeo llega al usuario.
- **Reintentar siempre y muchas veces**: convierte un servicio caído en **más carga sobre un servicio caído**, y multiplica la espera del usuario por el número de intentos.
- **Tres intentos con 200 ms de espera**, que es lo de aquí: cubre el parpadeo y acota el daño.

Y una línea que importa más de lo que parece: cuando el circuito del paso 4 esté abierto, el reintento **no debe reintentar**. Reintentar un rechazo instantáneo es esperar 400 ms para recibir tres veces el mismo «no» — y borra la ganancia del paso siguiente. Un circuito abierto no es un fallo transitorio: es **una decisión ya tomada**.

Se pega

En `practica/src/main/java/cl/dgt/resiliencia/services/PagoService.java`, **arriba**:

```
import io.github.resilience4j.retry.Retry;
import io.github.resilience4j.retry.RetryConfig;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

import java.time.Duration;
```

Los campos, **entre los de la clase**, y el bloque **dentro del constructor**, después de `this.cliente = cliente;`:

```
.maxAttempts(3)
.waitDuration(Duration.ofMillis(200))
.ignoreExceptions(CallNotPermittedException.class)
.build();
```

Vas bien si...

Con Tesorería caída ves en la consola las líneas >>> REINTENTO n.º 1 y >>> REINTENTO n.º 2 antes de que la petición falle.

Si te atascas

1 · No aparece ninguna línea de reintento.

Falta el publicador de eventos, o el `log` no está declarado. Sin la línea en consola el reintento puede estar funcionando igual — pero este lab se demuestra viéndolo.

2 · Ahora la petición tarda mucho más que 2 segundos.

Es lo esperado: son tres intentos de 2 s más las esperas entre ellos. El timeout limita **cada intento**, no el total.

3.4 Paso 4 · Dejar de llamar — el circuito

Qué vamos a hacer

Que, después de varios fallos seguidos, la aplicación **deje de llamar** a Tesorería durante un rato. Y contar las llamadas reales a la red.

Para entenderlo mejor

El diferencial del cuadro eléctrico. Cuando algo va mal, **corta**, y no se vuelve a subir hasta pasado un rato. No es que se rinda: es que insistir empeora las cosas.

Tres estados, y los vas a ver los tres:

Estado	Qué hace
CLOSED	todo normal, las llamadas pasan. Va contando fallos
OPEN	demasiados fallos: corta . Las llamadas fallan al instante, sin tocar la red
HALF_OPEN	pasado el rato, deja pasar unas pocas de prueba. Si van bien, cierra; si no, vuelve a abrir

El problema

Con Tesorería caída, cada petición que le llega **espera dos segundos y falla**. Con cien peticiones son doscientos segundos de hilos ocupados esperando algo que ya sabes que no va a llegar. Y encima, cada intento tuyo es una petición más contra un servicio que está intentando levantarse.

La alternativa, y por qué no

- **Sólo timeout y reintento:** fallas rápido, y sigues llamando eternamente a un muerto.
- **Los valores de fábrica del circuito:** pide **100 llamadas** antes de opinar. En una sesión de clase no abriría nunca; en un servicio con poco tráfico, tampoco.
- **Una ventana de 5 llamadas y 50 % de fallos**, que es lo de aquí: abre a tiempo y se puede ver en clase. Los cinco números están escritos a la vista para poder explicarlos; en un proyecto de verdad irían al `application.yml`, donde se cambian sin recompilar.

Se pega

Arriba, con los `import`:

```
import io.github.resilience4j.circuitbreaker.CallNotPermittedException;
import io.github.resilience4j.circuitbreaker.CircuitBreaker;
import io.github.resilience4j.circuitbreaker.CircuitBreakerConfig;

import java.util.function.Supplier;
```

La configuración, **dentro del constructor y delante del reintento**:

```
CircuitBreakerConfig configuracion = CircuitBreakerConfig.custom()
    .slidingWindowType(CircuitBreakerConfig.SlidingWindowType.COUNT_BASED)
    .slidingWindowSize(5)
    .minimumNumberOfCalls(5)
    .failureRateThreshold(50)
    .waitDurationInOpenState(Duration.ofSeconds(10))
    .permittedNumberOfCallsInHalfOpenState(2)
    .build();

this.circuito = CircuitBreaker.of("tesoreria", configuracion);
```

Y los dos métodos que permiten mirarlo desde fuera:

```
public Map<String, Object> metricas() {
    CircuitBreaker.Metrics m = circuito.getMetrics();
    return Map.of(
        "estado", circuito.getState().name(),
        "llamadasFallidas", m.getNumberOfFailedCalls(),
        "llamadasExitosas", m.getNumberOfSuccessfulCalls(),
        "tasaDeFallo", m.getFailureRate(),
        "llamadasHTTPReales", cliente.llamadasHechas());
}
```

Se corre

```
curl -X POST localhost:8097/simulador/caida
for i in 1 2 3 4 5 6 7; do
    curl -s -o /dev/null -w "%{time_total}s " localhost:8097/pagos/77
    curl -s localhost:8097/pagos/estado-circuito; echo
done
```


Lo que vas a ver

```
petición 1: 0.445439s  circuito=CLOSED  httpReales=3
petición 2: 0.220409s  circuito=OPEN   httpReales=4
petición 3: 0.001903s  circuito=OPEN   httpReales=4
petición 4: 0.001633s  circuito=OPEN   httpReales=4
petición 5: 0.001526s  circuito=OPEN   httpReales=4
petición 6: 0.002644s  circuito=OPEN   httpReales=4
petición 7: 0.001807s  circuito=OPEN   httpReales=4
```

Aquí hay que pararse y señalar el contador con el dedo.

httpReales es cuántas veces se salió de verdad a la red. **Se queda en 4 y no se mueve.** Se hicieron siete peticiones y sólo cuatro tocaron Tesorería: las otras tres se resolvieron aquí dentro, **en dos milésimas** — más de cien veces más rápido — y sin poner un gramo más de peso sobre un servicio que está intentando levantarse.

Ése es el doble regalo del circuito: protege a quien llama y a quien es llamado.

Y la vuelta:

```
curl -X POST localhost:8097/simulador/sana
sleep 11
curl -s localhost:8097/pagos/estado-circuito
```

```
{"estado":"HALF_OPEN","llamadasExitosas":1,"llamadasHTTPReales":1, ...}
```

HALF_OPEN: dejó pasar una de prueba, salió bien, y está a punto de cerrar.

Tus tiempos y tus contadores van a ser distintos, y el número exacto de peticiones hasta que abre también. Lo que tiene que verse es el patrón: **el tiempo se desploma y httpReales deja de subir.**

Vas bien si...

httpReales **se queda fijo** mientras el circuito está OPEN, y las peticiones pasan a tardar milésimas.

Si te atascas

1 · El circuito nunca abre.

Los valores de fábrica piden 100 llamadas. Comprueba que pusiste `slidingWindowSize(5)` y `minimumNumberOfCalls(5)`.

2 · Abre, pero las peticiones siguen tardando cientos de milésimas.

Falta el `.ignoreExceptions(CallNotPermittedException.class)` en el reintento: está reintentando el rechazo del circuito.

3 · httpReales sigue subiendo con el circuito abierto.

El circuito no está envolviendo la llamada, o lo pusiste **por fuera** del reintento en vez de por dentro.

3.5 Paso 5 · Contestar aunque no haya respuesta

Qué vamos a hacer

Devolver una respuesta con sentido cuando Tesorería no contesta.

Para entenderlo mejor

«No he podido confirmar el pago. Su trámite sigue su curso.» Es lo que diría una persona en la ventanilla, y es verdad. Un 500 diría «se rompió algo», que no es verdad: lo que pasa es que **no se sabe el estado del pago**.

El problema

Hasta ahora, con Tesorería caída, el usuario recibía un error. Rápido, pero un error — y el trámite que quería hacer **no depende de que Tesorería esté viva**.

La alternativa, y por qué no

- **Dejar salir el error (500)**: honesto en lo técnico y falso en lo funcional.
- **Un 503 con Retry-After**: correcto cuando el cliente **puede** reintentar y la respuesta a medias no le sirve de nada.
- **El último valor conocido, de una caché**: lo mejor de los tres **cuando hay algo que cachear**. Aquí no lo hay: el estado de un pago cambia.
- **Degradar**, que es lo de aquí: se responde 200 con un estado DESCONOCIDO y un aviso. **Es una decisión de negocio**, no técnica — y por eso la toma quien conoce el trámite, no quien escribe el cliente HTTP.

Se pega

Reemplazando el método consultar entero:

```
/** Paso 5: si no hay respuesta, se responde igual – con lo que se sabe. */
public Map<String, Object> consultar(String id) {
    Supplier<Map<String, Object>> protegida =
        Retry.decorateSupplier(reintento,
            CircuitBreaker.decorateSupplier(circuito, () ->
                cliente.consultarPago(id)));
    try {
        return protegida.get();
    } catch (Exception e) {
        log.warn("Treasurería no respondió ({}). Se degrada.",
            e.getClass().getSimpleName());
        return Map.of("estado", "DESCONOCIDO", "id", id,
            "aviso", "Treasurería no responde; el trámite sigue su curso");
    }
}
```

Lo que vas a ver

```
curl -X POST localhost:8097/simulador/caida
curl -s localhost:8097/pagos/77
```

```
{"estado":"DESCONOCIDO","id":"77","aviso":"Tesorería no responde; el trámite sigue su  
↪ curso"}
```

200, con la verdad dentro. El trámite puede seguir.

Vas bien si...

Con Tesorería caída, la respuesta es un 200 con "estado":"DESCONOCIDO" en vez de un error.

Si te atascas

1 · Sigue devolviendo un error.

El catch no está capturando lo que llega. Con el circuito abierto, la excepción es `CallNotPermittedException`, que no es un fallo HTTP: si capturas sólo excepciones de red, se te escapa.

4 Lo que aprendiste

1 · Sin timeout, quien decide cuánto esperas es el otro.

30,15 s contra 2,21 s, sobre el mismo servicio lento. El criterio no es cuánto tarda normalmente: es cuánto está dispuesto a esperar tu usuario.

2 · Reintentar cubre el parpadeo, y sólo el parpadeo.

Tres intentos salvan un paquete perdido y no salvan un servicio caído — ahí sólo multiplican la espera y la carga sobre el que está mal.

3 · El circuito protege a los dos lados.

Con él abierto, las llamadas se resolvieron en **0,0018 s** y el contador de llamadas reales **dejó de subir**. Tu aplicación deja de esperar, y el servicio caído deja de recibir peticiones mientras se levanta.

4 · Qué contestar cuando no hay respuesta es una decisión de negocio.

Degradar sólo es correcto si el trámite puede seguir sin ese dato. Eso no lo sabe quien escribe el cliente HTTP: lo sabe quien conoce el trámite.

5 Para profundizar

- **Sube el timeout a 10 segundos** y repite el paso 2. ¿Mejora algo?
- **Quita el `ignoreExceptions`** y mide otra vez con el circuito abierto. Compara con los 0,0018 s.
- **Baja `waitDurationInOpenState` a 3 segundos** y observa el paso por `HALF_OPEN`.
- **Pon el reintento por dentro del circuito** en vez de por fuera. ¿Qué le pasa al circuito ahora, con los reintentos contando como fallos?

6 Antes de cerrar

Párala con `Ctrl+C`, y deja Tesorería sana por cortesía con el próximo:

```
curl -X POST localhost:8097/simulador/sana
```

```
./mvnw clean
```

Lo que te llevas:

Timeout para no esperar indefinidamente, reintento para el parpadeo, circuito para dejar de llamar a un caído, y degradación para contestar igual. Las tres primeras son técnicas; la cuarta la decide el negocio.

Lo que queda pendiente, y abre el Lab 11: todo lo que has medido hoy lo has medido **tú, a mano, con un curl y un cronómetro**. En producción nadie está mirando. En el Lab 11 la aplicación empieza a contar lo que le pasa, y a decir si está en condiciones de atender.